Create your first idle game

This tutorial builds a small playable loop: buy a business, produce Coins, automate it, display the result, and save progress.

Table of contents

Before you begin

You need:

- TextMeshPro essentials imported if you use the included uGUI displayers.
- A scene with an EventSystem and Canvas for uGUI input.
- A unique, stable name for every persisted ScriptableObject.

Inspect the finished example

Open one of these scenes before building your own:

- Minimal loop: `EasyIdleGame > Demos > FeatureDemos > 01_BasicFlow > BasicFlow.unity`
- Larger guided example: `EasyIdleGame > Demos > DocsDemo > DocsScene.unity`

or find out more in the Demo catalog (`06-demo-catalog.pdf`).

Create the scene managers

1. Create an empty GameObject named `EasyIdleGame Managers` .
2. Add `CurrencyManager` from **Add Component > EasyIdleGame > Managers > Currencies Manager**.
3. Add `BusinessesManager` from **Add Component > EasyIdleGame > Managers > Businesses Manager**.
4. Add `ManagersManager` from **Add Component > EasyIdleGame > Managers > Managers Manager**.
5. Keep this GameObject enabled while gameplay runs.

`CurrencyManager` and `BusinessesManager` are required. Accessing either singleton without a scene component throws an exception.

If the scene will be replaced during play, keep this GameObject alive or ensure the next scene has exactly one correctly initialized manager set.

Create a Resources folder

Create this folder in your own game area:

`Assets/YourGame/Resources/Economy`

The folder can be nested anywhere, but one ancestor must be named `Resources` . The save system restores assets by name through `Resources.LoadAll` , so do not rename these assets after players have save files.

Create the Coin currency

1. Right-click the `Economy` folder.
2. Choose **Create > EasyIdleGame > Currencies > Currency**.
3. Name the asset `Coin`.
4. In `metadata`, set the display name to `Coins` and assign an icon if desired.
5. Keep the maximum amount unlimited unless your design needs a wallet cap.

The asset defines the currency. Its current and lifetime amounts live in a `CurrencyHolder` owned by `CurrencyManager`.

Set the Coin's display metadata freely, but keep the asset name stable once players can save the game: the display name may change without changing the saved identity, while the asset name is the save identifier.

Create the Lemonade Stand business

1. Right-click the `Economy` folder.
2. Choose **Create > EasyIdleGame > Businesses > Business**.
3. Name the asset `LemonadeStand`.
4. Set `metadata.name` to `Lemonade Stand`.
5. Under **Buying**, add a cost input:
   - `currencyInput`: `Coin`
   - `inputAmount`: `1`
   - `costMultiplier`: `1.15` for increasing prices, or `1` for a flat price
6. Under **Production**, set `timeToProduce` to `3`.
7. Add one output table:
   - Strategy: **Independent (All)**
   - Add one output with `currencyOutput = Coin`, `outputAmount = 1`, and `dropChance = 1`
8. Enable `autoCollect`.
9. Leave `autoProduce` disabled for now so the main button starts production manually.

Use `Single Round Live Amount` for the simplest first business. New copies bought during an active round join that round. The other round modes are covered in the core reference.

Add manager automation

1. Create **Create > EasyIdleGame > Managers > Manager** in the same Resources tree.
2. Name it `LemonadeManager`.
3. Set its metadata and add a Coin cost.
4. Set `autoProduceOnLevel` to `0` to automate as soon as the manager is bought, or to a positive manager level.
5. Set `autoCollectOnLevel` in the same way; `0` enables collection as soon as the manager is bought.
6. Assign `LemonadeManager` to the `manager` field on `LemonadeStand`.

An unlocked manager can automate the assigned business and supply passive or active `UpgradeBlock` modifiers. The manager asset does nothing until `ManagersManager` exists and the player owns/unlocks the manager at the required level.

Build the UI

The included uGUI displayers are optional helpers. You can use them directly, subclass them, or call the runtime API from your own UI.

1. Add a `CurrencyDisplayer` and assign `Coin`.
2. Add a `BusinessDisplayer` and assign `LemonadeStand`.
3. Add a `ManagerDisplayer` and assign `LemonadeManager`.
4. Connect buttons if your prefab does not already wire them:
   - Business purchase to `BusinessDisplayer.BuyBusiness()`.
   - Business action to `BusinessDisplayer.Click()`.
   - Manager purchase to `ManagerDisplayer.Buy()`.

    - Manager upgrade to `ManagerDisplayer.TryUpgradeManager()`.
    - Manager active ability, if configured, to `ManagerDisplayer.ActivateManager()`.
5. If TextMeshPro material appearance differs from the demo, import TMP essentials and adjust the material. The demo materials use outline `0.42`, and the demo layouts target a portrait aspect ratio of at least `9:16`.

For custom UI, obtain holders through the managers and subscribe to amount-change events instead of polling every field.

Add save and load

1. Add **EasyIdleGame > Managers > Save and Load** to the manager GameObject.
2. Set `savePath` to a unique slot name; the file is written to `Path.Combine(Application.persistentDataPath, savePath + ".json")`.
3. Set `maxOfflineSeconds` to the maximum time the game should simulate after load. The default is 86,400 seconds.
4. Load once after all manager components are available and before gameplay mutates state.
5. Call `SaveAndLoad.Save()` from important lifecycle points. The component also saves immediately on its first `Update`, then on its configured interval, so loading in `Start` must happen first.

The current loader automatically calculates and applies offline profit after a valid file is restored. You do not need to call `ProfitCalculator` a second time for the same elapsed period.

Load or seed the starting balance

The business costs one Coin, so a new save needs a starting balance. Add this project-side bootstrap component, attach it to the manager GameObject, and assign `Coin`:

```
using EasyIdleGame;
using UnityEngine;

public sealed class IdleGameBootstrap : MonoBehaviour
{
    [SerializeField] private Currency startingCurrency;
    [SerializeField] private BigNumber startingAmount = 10;

    private void Start()
    {
        SaveAndLoad.Instance.Load(out SaveFile file);

        if (file == null)
        {
            CurrencyManager.Instance.AddCurrencyAmount(
                startingCurrency,
                startingAmount,
                SourceType.Generic);
        }
    }
}
```

This loads existing progress before the first auto-save. It grants starting Coins only when no save file exists. Do not also call `Load()` elsewhere for the same launch. (You can also use Location & LocationManager to set starting currency.)

Verify the result

Enter Play Mode and confirm:

- The Coin and business displayers render without missing references.
- A new save starts with 10 Coins; an existing save restores its previous balance instead.

- Buying a Lemonade Stand removes the correct Coin cost.
- Clicking starts production and grants Coins after three seconds.
- Buying/unlocking the manager automates the business at the configured level.
- Saving creates JSON at `SaveAndLoad.Path`.
- Reloading restores the holder amounts and applies offline progress once.

If a definition cannot be restored, confirm that it is below a Resources folder and that no same-type asset shares its name.

Next steps

- Add costs, stockpiles, random drops, chains, and concurrent rounds with Core economy systems (`02-core-economy-systems.pdf`).
- Add upgrades, boosts, shops, offers, locations, rewards, and achievements with Feature guides (`03-feature-guides.pdf`).
- Plan save compatibility and resets with Persistence, offline profit, and prestige (`04-persistence-offline-prestige.pdf`).
- Integrate custom code through the Scripting reference (`05-scripting-reference.pdf`).
- Use a focused sample from the Demo catalog (`06-demo-catalog.pdf`).
- Diagnose setup problems with Troubleshooting (`07-troubleshooting.pdf`).